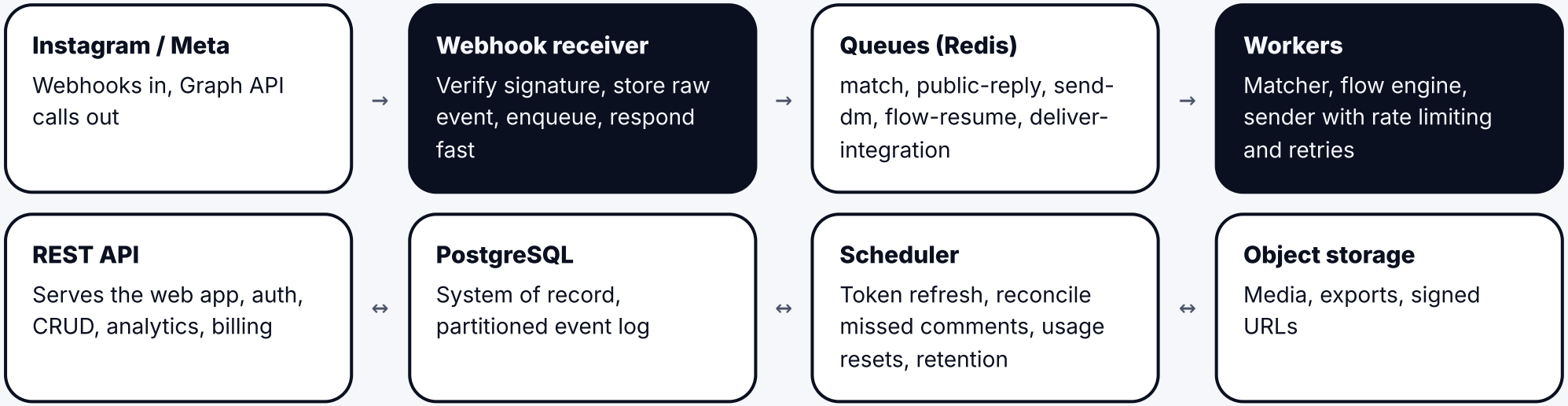
Backend specification

Everything the backend team needs to build the service behind the Instagram comment-to-DM tool: architecture, Instagram integration, data model, the event pipeline, flow engine, API, billing, security and operations. Product behaviour (pages, features, rules) is in the separate product specification; this document says how to build it.

Verify against Meta's current docs before building. Endpoint paths, permission names, field names, error codes and limits below come from general knowledge of the Instagram messaging and comments APIs and were not checked live for this draft. Pin an API version, and treat every "[confirm]" as a task for the first week.

01 System overview



Deployables

- **api:** stateless HTTP
- **webhooks:** tiny, independent, scaled for bursts
- **worker:** queue consumers, scaled by queue depth
- **scheduler:** single instance with leader lock

One repo (monorepo), shared packages, separate processes.

Stack

- Node.js LTS, TypeScript strict, NestJS or Fastify
- PostgreSQL 16, Prisma or Drizzle migrations
- Redis 7 with BullMQ
- Zod schemas, OpenAPI generated from them
- pino logs, OpenTelemetry, Sentry
- Docker, GitHub Actions, Terraform or equivalent

Design rules

- Webhook handler does no business logic
- Every job is idempotent
- Every skipped or failed message has a stored reason code
- Never call Instagram from a request path; always from a worker
- One source of truth for matching and rendering, shared with the frontend package

02 Instagram integration

Connect and tokens

STEP	DETAIL
1 Authorise	Redirect the user to Instagram login with scopes for basic profile, comments, messages and (V2) insights and content publishing [confirm exact scope names]. Use a signed, single-use `state` bound to the user session.
2 Exchange	Swap the code for a short-lived token, then for a long-lived token (about 60 days) [confirm]. Fetch the Instagram account id, username, account type and profile picture.

3 Validate	Reject non-professional accounts with a specific error code the frontend maps to the "switch account type" page.
4 Store	Encrypt the token with envelope encryption (data key per row, master key in KMS). Record granted permissions, expiry and status.
5 Subscribe	Subscribe the account to the webhook fields we need (comments, messages, postbacks, and later mentions and story replies) [confirm field names].
6 Refresh	Scheduler refreshes tokens that expire within [N] days. On failure mark the account `needs_reauth`, pause its automations with reason `TOKEN_INVALID`, notify the user.
7 Disconnect	On user action or Instagram deauthorise callback: revoke, pause automations, keep or delete data per user choice.

Webhooks

- **Verification (GET):** echo the challenge when the verify token matches.
- **Delivery (POST):** check the HMAC-SHA256 signature header against the raw body using the app secret, with constant-time comparison. Reject otherwise.
- Persist the raw payload (table `webhook_events`) with a content hash, enqueue one job per change, return 200 within a second.
- Expect duplicates, out-of-order and late events; never trust arrival order.
- Replay tool: re-enqueue stored events by id or time range.
- Reconcile job every [N] minutes: for live automations, list recent comments through the API and enqueue any not already seen, to cover dropped events.

```
// Illustrative comment event (confirm shape)
{
  "object": "instagram",
  "entry": [
    {
      "id": "<IG_ACCOUNT_ID>",
      "time": 1760000000,
      "changes": [
        {
          "field": "comments",
          "value": {
            "id": "<COMMENT_ID>",
            "text": "GUIDE please",
            "from": { "id": "<USER_ID>", "username":
"someone" },
            "media": { "id": "<MEDIA_ID>" }
          }
        }
      ]
    }
  ]
}
```

API calls we make [confirm]

PURPOSE	CALL	NOTES
Profile	`GET /me?fields=user_id,username,account_type,profile_picture_url`	On connect and daily refresh

List media	`GET /{ig-id}/media?fields=id,caption,media_type,media_product_type,permalink,thumbnail_url,timestamp`	Cursor pagination; thumbnail URLs expire, so copy to our storage
List comments	`GET /{media-id}/comments`	Reconcile job and catch-up
Public reply	`POST /{comment-id}/replies`	Separate from the DM, logged separately
Private reply (first DM)	`POST /{ig-id}/messages with recipient comment_id`	One per comment, within the allowed days after the comment, text only
Follow-up DM	`POST /{ig-id}/messages with recipient id`	Only inside the messaging window after the user replies or taps
User info and follow state	`GET /{igsid}?fields=name,username,profile_pic,is_user_follow_business`	Powers the Follow check block
Hide comment (V2)	`POST /{comment-id}?hide=true`	Optional after DM
Insights (V2)	`GET /{ig-id}/insights, /{media-id}/insights`	Rate-limited, cache aggressively
Publish (V2)	`POST /{ig-id}/media, then /media_publish`	Container then publish; poll status

Error handling

CLASS	EXAMPLES [CONFIRM CODES]	ACTION
Auth	Invalid or expired token, revoked permission	No retry. Mark account needs_reauth, pause automations, notify.

Rate limit	App or user level throttling	Retry with exponential backoff and jitter, lower the account's send rate, show "Delayed".
Temporary	5xx, timeouts, network	Retry up to [N] times, then dead-letter.
Permanent per message	Comment deleted, reply window passed, user cannot receive messages, message rejected for policy	No retry. Store reason code, show plain explanation.
Policy block	Account or app restricted	Pause the account's sending, alert staff, show banner.

03 Event pipeline

1 Ingest

Receiver stores `webhook_events`, enqueues `match` with the event id.

2 Match

Load live automations for the account and media. Evaluate keyword rules and exclusions, top-level only, schedule window, blocked users, opt-out, frequency rule. Write a `trigger_events` row with outcome.

3 Plan

Check plan usage and daily caps. Pick a public reply variation. Compute a send time with random delay and the account's rate limiter. Enqueue delayed jobs.

4 Send

Public reply job and DM job run independently. Update outcome, usage counters, contact fields. Handle postbacks by resuming the flow.

Guarantees

- **Idempotency key:** `igAccountId:commentId:automationId`, unique in the database and used as the queue job id. A duplicate webhook cannot create a second DM.
- **One private reply per comment** enforced by a unique constraint on `(ig_account_id, comment_id, kind)`.
- **Per-account rate limiting** with a Redis token bucket; concurrency of 1 or 2 per account to keep order and avoid bursts.
- **Daily and plan caps** use a Redis counter with periodic reconciliation to Postgres.
- **Dead-letter queue** with an admin replay screen.

Reason codes (stored per trigger event)

Skipped: `NO_KEYWORD_MATCH, EXCLUDED_WORD, ALREADY_SERVED, OUTSIDE_SCHEDULE, DAILY_CAP, PLAN_LIMIT, BLOCKED_USER, OPTED_OUT, NOT_TOP_LEVEL, PAUSED`

Failed: `TOKEN_INVALID, RATE_LIMITED, COMMENT_DELETED, REPLY_WINDOW_PASSED, USER_UNREACHABLE, POLICY_REJECTED, UNKNOWN`

The frontend maps each code to copy. Never return raw Instagram error text to the UI.

04 Flow engine

Model

- An automation points to a versioned flow: ordered steps with optional branches.
- Starting a flow creates a `flow_run` for a contact with `current_step`, `variables` (answers, tags), `window_expires_at`, status (active, waiting, completed, cancelled).
- The executor runs steps until it reaches one that needs input (button tap, quick reply, free text) or a delay, then parks the run.
- Incoming postbacks and messages find the waiting run by contact and resume it.
- Delays are delayed jobs and are rejected at publish time if they would fall outside the messaging window.

Step handlers

- **text, buttons, quick_replies, image:** render with variables, validate platform limits, send.
- **question:** park, validate answer (email, phone), store on contact, re-ask once on invalid.
- **follow_check:** read follow state, branch yes or no, limit nudges to one.
- **tag:** add or remove tag.
- **condition, delay, handoff, webhook (V2):** pure functions plus side effects through the outbox.
- Human takeover cancels active runs for that contact until resumed.
- Rendering and validation live in a shared package, with the same tests used by the frontend.

05 Data model

```
CREATE TABLE ig_accounts (
  id uuid PRIMARY KEY,
  workspace_id uuid NOT NULL,
  ig_user_id text NOT NULL UNIQUE,
  username text NOT NULL,
  token_ciphertext bytea NOT NULL,
  token_expires_at timestamptz,
  permissions text[] NOT NULL DEFAULT '{}',
  status text NOT NULL,          -- active, needs_reauth,
disconnected
  last_webhook_at timestamptz
);

CREATE TABLE automations (
  id uuid PRIMARY KEY,
  ig_account_id uuid NOT NULL REFERENCES ig_accounts,
  name text NOT NULL,
  status text NOT NULL,          -- draft, live, paused,
error
  trigger jsonb NOT NULL,
  keyword_rules jsonb NOT NULL,
  public_replies jsonb NOT NULL,
  flow_id uuid NOT NULL,
  settings jsonb NOT NULL,
  created_by uuid, updated_at timestamptz
);

CREATE TABLE trigger_events (
  id uuid,
  ig_account_id uuid NOT NULL,
  automation_id uuid,
  contact_id uuid,
```

Other tables

- **workspaces, users, memberships** (role)
- **media:** synced posts, reels, stories with copied thumbnail
- **flows, flow_steps** (versioned) and **flow_runs**
- **contacts:** igsid, username, name, email, phone, tags, follows, opted_out, first_seen, last_seen; unique by (ig_account_id, igsid)
- **conversations, messages** (direction, source bot or human, ig_message_id)
- **lead_answers**
- **templates** (system and workspace)
- **link_clicks:** short code, contact, time
- **webhook_events:** raw payload, hash, processed_at
- **usage_counters:** workspace, period, dms_sent
- **subscriptions, invoices** (provider ids)
- **integrations, integration_deliveries** (outbox)
- **audit_log, notifications**
- **stats_hourly:** rollups per automation and media

CONVENTIONS

- UUIDs, timestamps in UTC, soft delete only where needed
- Partition trigger_events and webhook_events by month; drop or archive by retention
- Index for the main queries: by account and time, by automation and time, contacts search

```
  comment_id text NOT NULL,
  comment_text text,
  matched_keyword text,
  outcome text NOT NULL,         -- sent, skipped, failed,
queued
  reason_code text,
  attempts int NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL,
  sent_at timestamptz,
  PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (created_at);

CREATE UNIQUE INDEX ON trigger_events
  (ig_account_id, comment_id, automation_id, created_at);
```

06 REST API

JSON over HTTPS under `/v1`. Cursor pagination (`?cursor=&limit=`). `Idempotency-Key` header on POST. Errors use one shape: code, message, field errors, request id. OpenAPI is generated and published for the frontend to generate its client. Rate limit per user and IP.

RESOURCE	ENDPOINTS	NOTES
Auth	`POST /auth/signup, /login, /logout, /reset; GET /me; GET /auth/instagram/start, /callback`	Session cookie, httpOnly, SameSite, CSRF token
Accounts	`GET /ig-accounts; POST /ig-accounts/:id/reconnect; DELETE /ig-accounts/:id`	Includes token health and permissions
Media	`GET /media; POST /media/sync; GET /media/:id`	Filter by type and automation

Automations	`GET, POST /automations; GET, PATCH, DELETE /automations/:id; POST /:id/publish, /pause, /resume, /duplicate, /test`	PATCH accepts partial drafts, validated by shared schema; publish runs full validation
Activity	`GET /automations/:id/events; POST /events/:id/retry`	Cursor, filter by outcome and reason; retry only when allowed
Templates	`GET /templates; POST /templates (save from automation); DELETE`	System templates are read-only
Contacts	`GET /contacts; GET, PATCH, DELETE /contacts/:id; POST /contacts/export; POST /contacts/bulk-tag`	Delete removes personal data everywhere; export is a background job
Analytics	`GET /analytics/overview, /by-automation, /by-media, /keywords, /heatmap`	Date range, compare, tags; reads rollups
Billing	`GET /billing; POST /billing/checkout, /portal; GET /billing/invoices; GET /usage`	Provider hosted checkout
Workspace	`GET, PATCH /workspace; /members, /invites (V2)`	Role checks centrally
Inbox V2	`GET /conversations; GET /:id/messages; POST /:id/messages; POST /:id/takeover, /resume; SSE /stream`	Reply blocked when window closed
Integrations, Reports, Planner V2	`/integrations, /reports, /posts`	Defined when those features start
Public and internal	`GET /l/:code (click redirect); POST /meta/data-deletion; GET, POST /webhooks/instagram; POST /webhooks/payments`	No session; verified by signature

Admin V2	`/admin/*`	Separate auth, staff roles, full audit

ROLE	CAN	CANNOT	PHASE
Owner	Everything, billing, delete workspace	-	MVP
Admin	Manage accounts, automations, team	Billing, delete workspace	V2
Editor	Build automations, contacts, inbox	Accounts, billing, team	V2
Viewer	Read analytics and contacts	Change anything	V2

07 Billing and usage

- Provider: Razorpay or Stripe [decide by market]; hosted checkout and customer portal, our database stores only provider ids and status.
- Plans and limits are configuration, not code: accounts, DMs per period, live automations, seats, features.
- 14-day trial [confirm] with the paid features on and a clear end date.
- Usage: each successful DM increments `usage_counters`. Warn at 80 percent, at 100 percent pause new sends with reason `PLAN_LIMIT` and notify; do not drop events, record them as skipped.
- Add-on "buy more DMs" packs credited to the current period.

- Payment webhooks are verified, idempotent and stored: created, activated, charged, failed, cancelled, refunded.
- Past due: grace period [N] days, then downgrade to Free limits and pause extra automations, never delete data.
- Invoices and tax details suitable for the target country (for example GST in India) [confirm with accountant].
- Admin can grant credits or override a plan, always audited.

08 Analytics and links

Rollups

- Worker maintains `stats_hourly` from trigger_events, messages and clicks
- Dashboards read rollups, details read the event log
- Backfill command to rebuild from events

Link tracking

- URL buttons point to `/l/:code`, which records the click and redirects
- Validate destination URLs, block unsafe schemes
- Clicks attributed to automation and contact

Exports

- CSV generated by a job, written to object storage, emailed or shown as a signed link that expires
- Stream rows, never load all in memory
- Respect deletion and column permissions

09 Security and privacy

Application

- Tokens and secrets encrypted, rotated, never logged
- Passwords with Argon2id or delegated to the auth provider
- Input validation on every route, output encoding in emails
- CORS to our origins only, CSRF protection for cookie sessions
- Outgoing webhook URLs checked against internal addresses (SSRF)
- Dependency and container scanning in CI

Data protection

- Meta data deletion callback: verify the signed request, delete the person's data, return a confirmation code and a status URL
- User-initiated deletion and export from Settings
- Retention setting per workspace; scheduled purge
- PII minimised: no comment text in analytics or error tracking
- Applicable law for target markets (for example India's DPDP Act, GDPR) [confirm with counsel]

Access and audit

- Role checks in one policy layer, tested
- Audit log for connect, disconnect, publish, export, delete, billing, admin actions
- Staff access through SSO and two-factor, time-limited impersonation with consent
- Backups with point-in-time recovery, restore tested quarterly

10

Observability and operations

Metrics and alerts

- Webhook receive rate and lag between event time and processing
- Queue depth and oldest job age per queue
- Comment-to-DM latency p50 and p95, and success rate
- Instagram error rate by class and code, per account and global
- Tokens expiring soon, accounts in needs_reauth
- Alert when: no webhooks for an active account for [N] hours, backlog above [N] for [N] minutes, error rate spike, payment webhook failures

Runbooks

- Instagram outage or elevated errors: pause sends, keep queueing, resume gradually
- Mass token failures after a Meta change
- Queue backlog: scale workers, check rate limiter, check one noisy account
- Duplicate DM report: inspect idempotency key and trigger_events
- Meta policy warning or account restriction

Environments

- Local, staging, production, each with its own Meta app, webhook URL, database and keys
- Test Instagram accounts for staging
- Migrations run in CI with backward-compatible steps
- Feature flags for V2 modules

Starting capacity (to size, not final)

- Webhook receiver: 2 small instances behind a load balancer
- Workers: 2 instances, autoscale on queue depth
- Postgres: managed, one primary plus replica later
- Redis: managed, persistence on
- Load target [N] comments per second per account burst, [N] per second overall

11 Testing

Unit

- Keyword matcher: table-driven cases for modes, punctuation, emoji, exclusions, languages
- Flow executor and renderer, window validation
- Policy and permission checks

Integration

- Mock Meta server returning success, throttling, expiry, deleted comment
- Duplicate, late and out-of-order webhooks
- Payment webhook replay

System

- k6 load test of webhook receiver and queues
- End-to-end on staging with real test Instagram accounts
- Restore and failover drills

12 Delivery plan

SPRINT	DELIVERABLES	FRONTEND NEEDS
0 Foundation	Monorepo, CI, environments, Postgres and Redis, auth, OpenAPI skeleton, Meta app created and review started	OpenAPI draft, mock server
1 Connect	Instagram OAuth, token encryption and refresh, account status, webhook receiver with verification and replay	Auth and accounts endpoints
2 Content	Media sync and thumbnail storage, templates data, contacts model	Media and templates endpoints
3 Automations	Automation CRUD with shared schema validation, publish, pause, keyword matcher	Draft API

4 Sending	Queues, rate limiter, public reply and private reply senders, idempotency, reason codes, test send	Test and activity endpoints
5 Flows	Flow engine with buttons, questions, follow check, tags; postback handling; contact updates	Flow step schema final
6 Insight	Rollups, analytics endpoints, link tracking, exports, retention jobs	Analytics and export endpoints
7 Money and launch	Billing, usage enforcement, notifications, data deletion callback, observability, load test, security review	Billing endpoints
V2	Inbox with realtime, story and DM triggers, sequences, integrations outbox, planner and publishing, insights, teams, admin console, catch-up	Per feature

13 Open questions

- Which Meta product path and permissions do we use, and has business verification been started? (Confirm first; it can gate the whole schedule.)
- Cloud and region: where do our target customers' data need to live?
- Payment provider and tax setup for the launch market.
- Do we build auth ourselves or use a provider?
- Message retention: how long do we keep comment text and DMs by default?
- Expected launch volume, so we can size queues and the rate-limit strategy.
- Do agencies and the inbox ship in the first release?